CSE 325

Razor Pages Movie Tutorial Easy Study Guide

A plain-English guide to the ASP.NET Core Razor Pages tutorial

Main idea: You built a CRUD website that stores movies in a database. Razor Pages handles the web pages, C# handles the logic, and Entity Framework Core connects your C# objects to the database.

Use this guide when the Microsoft tutorial feels like a lot of disconnected steps.

How to use this guide

- Read the "Big picture" first. It explains how all of the pieces connect.
- Use the individual sections when you forget what a file, command, or concept does.
- Use the final cheat sheets before quizzes, assignments, or when debugging.
- Do not try to memorize every generated line of code. Focus on the job each piece performs.

1. The big picture

The whole tutorial teaches one story:**User -> Razor Page -> PageModel -> DbContext / EF Core -> Database**

CRUD: Your movie app can Create, Read, Update, and Delete movie records. Search and validation are extra features built on top of that core CRUD workflow.

Layer	What it does	Example
Browser / User	Where the person sees and submits information	Movies list, Create form
Razor Page (.cshtml)	The page markup and UI	Index.cshtml
PageModel (.cshtml.cs)	The C# logic for that page	Index.cshtml.cs
EF Core / DbContext	Connects C# objects to database data	RazorPagesMovieContext
Database	Stores the movie records	SQLite database

2. Project anatomy: what the files mean

File or folder	Remember it as
Program.cs	Starts and configures the application
Models/Movie.cs	Defines what a Movie is
Data/RazorPagesMovieContext.cs	Database manager / bridge for Movie data
Pages/Movies/Index.cshtml	Movie list UI
Pages/Movies/Index.cshtml.cs	Movie list C# logic

Pages/Movies/Create.cshtml	Form for adding a movie
Pages/Movies/Edit.cshtml	Form for changing a movie
Pages/Shared/_Layout.cshtml	Shared navbar, page shell, footer
appsettings.json	Configuration, including connection strings
Migrations/	History and instructions for database schema changes
RazorPagesMovie.csproj	Project settings and NuGet package references

Key pairing: .cshtml is what the user sees; .cshtml.cs is the C# logic behind that page. Think of them as two halves of one Razor Page.

3. Program.cs: prepare, build, run

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddRazorPages();

var app = builder.Build();

app.MapRazorPages();
app.Run();
```

Code	Simple meaning
CreateBuilder(args)	Prepare configuration, services, logging, and hosting
builder.Services...	Register things the app will need
Build()	Assemble the configured web application
MapRazorPages()	Connect Razor Pages to routes/URLs
Run()	Start the server so it can receive requests

Restaurant analogy: CreateBuilder = prepare the restaurant. Build = everything is assembled and ready. Run = unlock the doors and start serving requests.

4. The Movie model

```
public class Movie
{
    public int Id { get; set; }
    public string Title { get; set; } = string.Empty;
    public DateTime ReleaseDate { get; set; }
    public string Genre { get; set; } = string.Empty;
    public decimal Price { get; set; }
    public string Rating { get; set; } = string.Empty;
}
```

The model is the shape of your data. You invented the type name Movie. Every Movie object follows this structure.

Model question: Ask: "What information does one movie need to contain?" The answer becomes properties on the Movie class.

5. Entity Framework Core and DbContext

Entity Framework Core (EF Core) is the bridge between your C# code and your database. It lets you work with C# objects instead of manually writing SQL for every operation.

Thing	Easy meaning
EF Core	Tool that translates between C# objects and database operations
DbContext	The object that manages database access for your app
DbSet<Movie>	Represents the collection/table of Movie records
_context.Movie	Your code's way to access the Movie data

Mental model: PageModel asks DbContext for data. DbContext uses EF Core. EF Core talks to the database.

6. Scaffolding: code generation

Scaffolding means: "Generate the repetitive CRUD code for me." It created the Movie pages and database context so you did not have to write every file from scratch.

Generated page	Job

Index	List movies
Create	Add a movie
Details	View one movie
Edit	Change a movie
Delete	Remove a movie

Important: Generated does not mean magical. You still need to understand what the generated code is doing, especially OnGet, OnPost, DbContext, and model binding.

7. GET, POST, and PageModel handlers

Handler	When it runs	Common job
OnGet / OnGetAsync	When a page is requested	Load data to display
OnPost / OnPostAsync	When a form is submitted	Validate and save submitted data

```
public async Task OnGetAsync()
{
   Movie = await _context.Movie.ToListAsync();
}
```

That means: when the Movies page is opened, get all Movie records from the database and put them in the Movie property so the page can display them.

```
public async Task<IActionResult> OnPostAsync()
{
   if (!ModelState.IsValid)
      return Page();

   _context.Movie.Add(Movie);
   await _context.SaveChangesAsync();
   return RedirectToPage("./Index");
}
```

That means: validate the form, add the Movie, save the database changes, then return to the list page.

8. Dependency injection

```
public IndexModel(RazorPagesMovieContext context)
{
   _context = context;
}
```

The page needs a database context. Instead of creating one itself, ASP.NET supplies one. That is dependency injection.

Restaurant analogy: A chef needs ingredients. The chef does not leave the kitchen to grow tomatoes. The restaurant supplies what the chef needs. ASP.NET supplies the services your class needs.

9. Migrations: keeping the model and database in sync

Changing Movie.cs does NOT automatically change the database. Migrations keep the C# model and database schema synchronized.

```
dotnet ef migrations add InitialCreate
dotnet ef database update
```

Command	Meaning
dotnet ef migrations add Name	Create a plan/instructions for the schema change
dotnet ef database update	Apply those migration instructions to the database

Memory trick: Migration = plan. Database update = execute the plan.

Typical workflow when the model changes:

1. Change Movie.cs
2. Save and build
3. Create a migration
4. Update the database
5. Run and test the app

10. Razor syntax, layout, and Tag Helpers

```
@page
@model RazorPagesMovie.Pages.Movies.IndexModel

@foreach (var item in Model.Movie)
```

```
{
  <p>@item.Title</p>
}
```

Syntax	What it means
@page	This .cshtml file can handle a request
@model	Tells the page which PageModel type it uses
@item.Title	Switch from HTML/Razor into C# to display a value
asp-page	Tag Helper that builds a Razor Page link
asp-for	Tag Helper that connects an input to a model property

_Layout.cshtml contains shared parts of the site such as the navigation bar and footer. Change it once, and every page using that layout reflects the change.

DRY: Do not repeat yourself. Shared layout and model validation let you define behavior in one place and reuse it everywhere.

11. Routes, IDs, and 404 errors

```
@page "{id:int}"
```

This says: the URL must contain an id, and that id must be an integer.

Template	Example	Meaning
{id:int}	/Movies/Details/1	ID is required and must be an integer
{id:int?}	/Movies/Details or /Movies/Details/1	ID is optional because of ?

Two kinds of 404: With {id:int}, routing can reject the URL before OnGet runs. With {id:int?}, OnGet can run with id = null and your code can return NotFound().

12. Search, LINQ, and lambda expressions

```
var movies = from m in _context.Movie
             select m;

if (!string.IsNullOrEmpty(SearchString))
{
   movies = movies.Where(s => s.Title.Contains(SearchString));
}

Movie = await movies.ToListAsync();
```

The query is built first, then filtered if the user entered search text, then finally executed when ToListAsync() runs.

Concept	Easy meaning
LINQ	C# tools for querying/filtering collections and database data
Where(...)	Keep only the items that match a condition
Lambda	A short function, such as s => s.Title.Contains(SearchString)
ToListAsync()	Execute the query and return the results as a list
Deferred execution	The query can be built/modified before it actually runs

Read the lambda: s => s.Title.Contains(SearchString) means: "For each movie s, keep it if its Title contains the search text."

13. Genre filtering and SelectList

SelectList is used to populate a dropdown list of genres. Distinct() removes duplicate genre names.

```
Genres = new SelectList(
   await genreQuery.Distinct().ToListAsync());
```

Code	Meaning
SelectList	Data used to build the dropdown options
Distinct()	Keep each genre only once

MovieGenre	The genre the user selected

14. Adding a new field

When you added Rating, you learned the full "change the data model" workflow.

6. Add the Rating property to Movie.cs.
7. Add Rating to Create, Edit, Details, Delete, and Index as needed.
8. Update seed data if the project uses SeedData.
9. Build the project.
10. Create a migration.
11. Run dotnet ef database update.
12. Run the app and test creating/editing/displaying Rating.

Why migrations matter: If Movie.cs expects Rating but the database table does not have a Rating column, the model and database are out of sync.

15. Validation and Data Annotations

```
[Required]
[StringLength(60, MinimumLength = 3)]
public string Title { get; set; } = string.Empty;

[Range(1, 100)]
public decimal Price { get; set; }
```

Attribute	Meaning
[Required]	A value is required
[StringLength]	Sets maximum and optionally minimum string length
[Range]	Numeric value must stay within a range
[RegularExpression]	Text must follow a required pattern
[Display]	Changes the display label
[DataType]	Provides formatting/semantic hints; not itself a validation rule

The key benefit: you define validation rules in Movie.cs and the Create/Edit pages can automatically use those rules.

```
if (!ModelState.IsValid)
{
    return Page();
}
```

That means: if the submitted data violates validation rules, do not save it. Redisplay the form with errors.

Two layers: Client-side validation gives fast feedback in the browser. Server-side validation checks again on the server and should always be trusted as the final check.

16. End-to-end flow: what happens when a user opens or submits a page?

Opening the Movies page:**Browser -> Route -> OnGetAsync -> DbContext -> EF Core -> Database -> PageModel -> Razor Page -> Browser**

Creating a movie:**Form -> POST -> Model binding -> Validation -> OnPostAsync -> DbContext -> SaveChangesAsync -> Database -> Redirect**

17. Common commands

Command	Use it for
dotnet build	Check whether the project compiles
dotnet run	Start the app
dotnet ef migrations add Name	Create a migration
dotnet ef database update	Apply migrations to the database
Ctrl+C	Stop a running app in the terminal

18. Common problems you ran into

Message / symptom	What it usually means	First thing to try
Address already in use	Another copy of the app is already using that port	Stop the other running app, then run again
File is locked by RazorPagesMovie	The executable is still running during a rebuild	Ctrl+C or stop the debugger

No DbContext was found	Scaffolding/context creation was not completed yet	Create/scaffold the DbContext before migrations
CS0246 SelectList could not be found	Missing namespace import	Add using Microsoft.AspNetCore.Mvc.Rendering;
NU1901 warning	A NuGet dependency has a known vulnerability	Treat as a warning; update packages separately
0 Error(s), warnings present	The project still built successfully	Continue unless the warning matters to the assignment

19. Vocabulary cheat sheet

Word	Easy definition
Razor Page	A web page built with Razor markup and C#
Model	Definition of what your data looks like
Entity	One object/record stored in the database
EF Core	Tool that maps C# objects to database operations
DbContext	Object that manages database access
CRUD	Create, Read, Update, Delete
Scaffolding	Automatically generates repetitive starter code
Migration	Instructions/history for changing database structure
Dependency injection	ASP.NET provides a class with the services it needs
Middleware	Code requests pass through in the request pipeline
Route	URL pattern used to reach a page
Model binding	Moves request/form/route data into C#

	properties/parameters
Lambda	Short function such as x => x.Title
LINQ	C# tools for querying/filtering data
Data Annotation	Attribute such as [Required] that describes/validates data
Namespace	Organizes types and avoids name conflicts
using	Lets you use types from a namespace without the full name

20. If you remember only 10 things

13. Razor Pages usually come in pairs: .cshtml for UI and .cshtml.cs for C# logic.
14. Movie.cs defines the shape of movie data.
15. EF Core connects your C# objects to the database.
16. DbContext is the main database access object your pages use.
17. Scaffolding generated the CRUD pages for you.
18. OnGet loads data; OnPost handles submitted forms.
19. A migration plans a database schema change; database update applies it.
20. LINQ and lambdas help filter/query the movie data.
21. Data Annotations define validation rules once in the model.
22. The whole app is one flow: user -> Razor Page -> PageModel -> EF Core/DbContext -> database -> back to the user.

One-paragraph summary: You created a Razor Pages website, defined a Movie model, used EF Core and DbContext to store Movie objects in a database, scaffolded CRUD pages, used OnGet and OnPost handlers for page requests and forms, used migrations to keep the database schema in sync with model changes, added LINQ-based search and genre filtering, and added Data Annotation validation so invalid movie data is not saved.